

LEXICOGRAPHIC MULTI-OBJECTIVE ON-POLICY DISTILLATION

Doseok Jang *

Cohere,
Mila, Université de Montréal,
doseok.jang@mila.quebec

Jon Ander Campos

Cohere
jonander@cohere.com

Youran Qi

Cohere
youran@cohere.com

ABSTRACT

Reinforcement learning from verifiable rewards (RLVR) usually optimizes answer correctness, yet useful language-model behavior also requires high-quality reasoning and concise responses. Existing multi-reward post-training methods typically scalarize rewards or combine specialists without explicitly protecting a reward priority order. This is problematic when trade-offs are asymmetric: conciseness, for example, should not improve at the cost of correctness. We introduce *Lexicographic Multi-Objective On-Policy Distillation* (LMOPD), a multi-teacher method for integrating reward-specialized policies under explicit priorities. For each student rollout, LMOPD selects the specialist for the first objective whose gate detects a deficiency, then locally projects its centered log-policy correction to remove components that oppose higher-priority specialists. We evaluate 30B-A3B mixture-of-experts transformer models in two- and four-expert settings on three math benchmarks, measuring retained specialist gains. With two experts, LMOPD’s point estimates fully retain the accuracy and reasoning-quality gains while acquiring 46.9% of the conciseness gain. With four experts, it retains $\approx 90\%$ of both the accuracy gain and reasoning-correctness gain, compared to only $\approx 57\%$ by the next best evaluated baseline. Matched four-expert ablations show that lexicographic routing outperforms random routing and that projection further strengthens both top-priority capabilities. Across both scales, LMOPD preserves the highest-priority capabilities more effectively than the existing baselines we evaluate, demonstrating the value of explicit priorities for specialist integration.

1 INTRODUCTION

Post-training has become a critical stage in building large language models (LLMs) that can solve difficult reasoning and agentic tasks. Policy-gradient methods such as Proximal Policy Optimization (PPO) (Schulman et al., 2017) and Group Relative Policy Optimization (GRPO) (Shao et al., 2024) enable models to improve from outcome-level feedback, and reinforcement learning from verifiable rewards (RLVR) has been especially effective in domains such as mathematics and code where correctness can be automatically verified. Modern reasoning systems consequently devote substantial post-training compute to reinforcement learning on verifiable tasks (Yang et al., 2025).

Outcome correctness, however, is only one dimension of response quality. A useful mathematical assistant should also reason correctly, verify its work, avoid unnecessary dead ends, and communicate concisely. These objectives can conflict and have reward signals with different scales and densities. A standard solution is scalarization; GDPO, for example, normalizes reward dimensions separately before combining them (Liu et al., 2026a). A fixed scalarization still treats objectives as compensatory: enough improvement on a lower-priority axis can offset degradation of a prerequisite. Many applications instead require a non-compensatory hierarchy in which presentation quality cannot redeem an incorrect solution.

An alternative is to specialize separate models and combine them after training. Rewarded Soups (Ramé et al., 2023), for example, interpolates parameters of models optimized for different rewards.

*Work done as an intern at Cohere

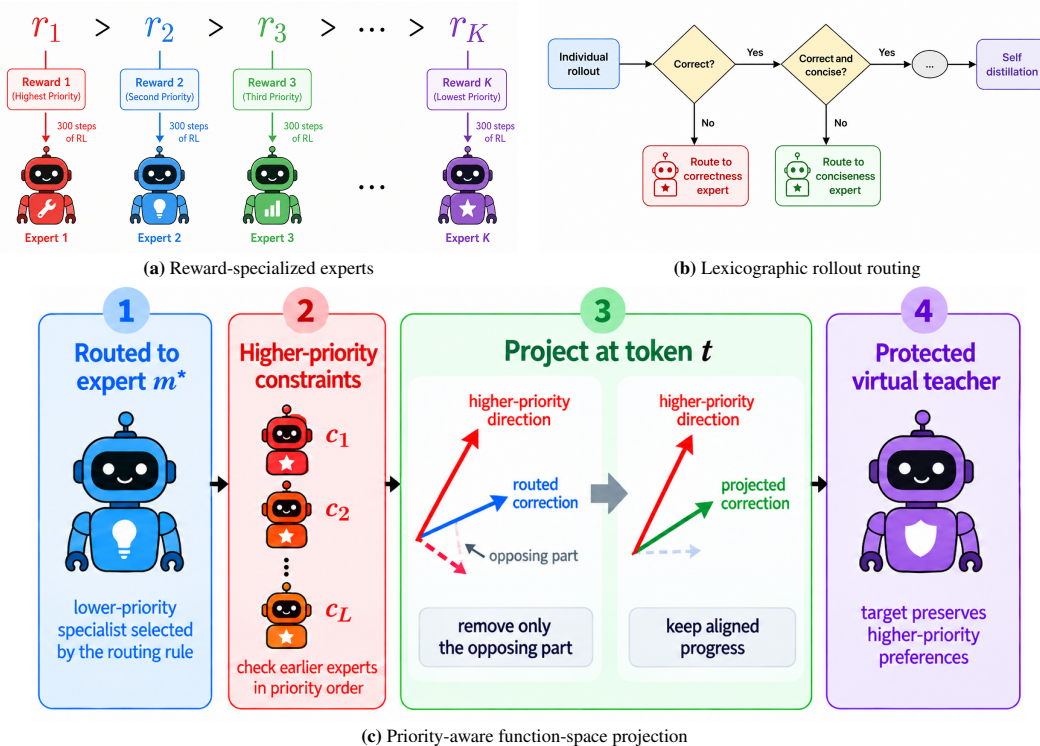


Figure 1: **LMOPD overview.** (a) We train reward-specialized experts and assign each reward to an expert; one expert may serve multiple rewards. (b) Each student rollout is checked in that order and routed to the first objective whose gate detects a deficiency; a rollout that passes every gate is self-distilled. (c) At each token, LMOPD removes only components of the routed specialist’s correction that oppose earlier specialists, producing a projected virtual-teacher target.

On-policy distillation (OPD) instead transfers dense teacher feedback on student-generated trajectories (Agarwal et al., 2024). Multi-teacher OPD (MOPD) routes rollouts to domain-specialized teachers (Ma et al., 2026); there, the prompt domain determines which teacher rollouts are routed to. We study when several objectives apply to the *same* prompt, where the missing signal is instead which requirement the individual sampled solution fails.

We propose **Lexicographic Multi-Objective On-Policy Distillation (LMOPD)** for priority-ordered specialist integration. LMOPD uses each rollout’s reward profile to make the routing decision. We assume an ordered set of objectives $r_1 \succ r_2 \succ \dots \succ r_K$ and a collection of reward-specialized experts. For each on-policy student rollout, LMOPD walks through the rewards in priority order. A discrete objective routes to its expert when its maximum acceptable reward has not been achieved; a continuous objective routes when the rollout has negative group-relative advantage on that objective. The first objective that triggers determines the teacher. If every objective passes, the rollout is routed to the student itself (effectively contributing no gradient) to avoid unnecessarily perturbing already correct behavior. Higher-priority objectives therefore have the first opportunity to correct a trajectory, while lower-priority experts only act after higher-priority requirements are satisfied.

Routing alone does not prevent a selected lower-priority expert from proposing a token-level correction that opposes a higher-priority expert. We therefore represent each expert’s correction as a centered log-policy delta and sequentially remove components with negative inner product against the higher-priority expert deltas. This projection is performed in function space rather than parameter-gradient space and is implemented through streamed per-token Gram matrices, avoiding one additional student backward pass per expert.

This paper makes three contributions:

- We formulate *lexicographic corrective routing* for multi-teacher OPD. The per-rollout rule handles absolute discrete criteria and relative continuous rewards, allows several objectives to share one expert, and self-distills when no correction is required.
- We introduce a one-sided function-space projection that removes components of the routed correction that oppose higher-priority experts, together with a streamed Gram-space implementation that requires no extra student backward passes.
- We evaluate LMOPD in two- and four-expert settings at 30B scale and isolate routing and projection through component ablations.

We evaluate whether LMOPD retains prerequisite capabilities, improves teacher assignment over random routing, and redirects integration when projection activates. LMOPD is a local supervision heuristic, not a claim of globally optimal lexicographic control.

2 RELATED WORK

RLVR and multi-reward reinforcement learning. PPO (Schulman et al., 2017) and GRPO (Shao et al., 2024) are widely used policy-gradient methods for LLM post-training. RLVR couples these updates with automatically verifiable outcome rewards, particularly for mathematical and coding tasks. Multi-reward extensions introduce additional reward dimensions for properties such as style, format, length, or process quality. GDPO (Liu et al., 2026a) identifies a failure mode in naively applying group normalization after summing heterogeneous rewards and instead normalizes each reward dimension independently before aggregation. This preserves reward-specific variation, but the final policy update still depends on a scalarized combination of objectives. Recent work also uses conditional reward structures. SpatialThinker explicitly uses lexicographically gated rewards in multimodal RL (Batra et al., 2025), while SCOPE-RL gates process-quality rewards on final-answer correctness in mathematical reasoning (Liu et al., 2026b). LMOPD shares the intuition that lower-priority feedback should not override task success, but uses this hierarchy to select a *teacher distribution* rather than to construct a scalar reward.

Combining reward-specialized policies. Rewarded Soups (Ramé et al., 2023) trains policies on different rewards from a shared initialization, linearly interpolates their weights, and uses the resulting merged model for all inputs and trajectories.

On-policy and multi-teacher distillation. Knowledge distillation trains a student to match a teacher distribution; reverse KL provides a mode-seeking signal for autoregressive LLMs (Gu et al., 2024). GKD uses student-generated sequences (Agarwal et al., 2024) instead, providing feedback that is both dense and on-policy. MOPD extends GKD by dispatching rollouts to domain-specialized teachers to mix capabilities across domains and multiple teachers (Ma et al., 2026). MOPD has emerged as a key component of frontier LLM post-training recipes, including MiMo-V2-Flash Xiao et al. (2026), DeepSeek-V4 Xu et al. (2026), GLM-5 Zeng et al. (2026), and Nemotron-Cascade 2 Yang et al. (2026). SMOPD, a concurrent work, first trains reward specialists, then merges a token-level mixture of all teachers with a balanced multi-reward policy-gradient anchor (Wang et al., 2026).

At the mechanism level, LMOPD is related to PCGrad, which removes pairwise conflicting parameter-gradient components in multi-task learning (Yu et al., 2020). LMOPD instead filters one routed teacher correction only against earlier priorities, using an unweighted token-level function-space surrogate rather than parameter gradients.

Lexicographic optimization. Lexicographic multi-objective reinforcement learning optimizes objectives according to an ordered preference relation rather than treating all trade-offs as interchangeable. Recent work studies this setting theoretically and algorithmically, including regret guarantees for linear MDPs (Xue et al., 2025) and thresholded lexicographic formulations (Tercan & Prabhu, 2024). LMOPD does not attempt to provide the same policy-level optimality guarantees. Instead, it borrows the priority structure to select a corrective teacher signal and locally project that signal during distillation.

Across these lines of work, LMOPD differs in three coupled choices: unlike domain-routed MOPD, it routes individual rollouts by reward deficiency; unlike Rewarded Soups and SMOPD, it does not globally or simultaneously merge all specialists; and unlike GDPO, it does not collapse priorities into a scalarized advantage.

3 METHOD

LMOPD has two conceptually separable components. First, *lexicographic rollout routing* uses the reward profile of each student-generated completion to select the specialist associated with its highest-priority deficiency. Second, *priority-aware projection* modifies the selected specialist’s token-level correction whenever it opposes a higher-priority specialist. This decomposition yields a routing-only method and a routing-plus-projection method with the same rollout assignments and distillation loss. Figure 1 summarizes the complete pipeline.

3.1 LEXICOGRAPHIC MULTI-OBJECTIVE SETTING

Let $x \sim \mathcal{D}$ denote a prompt and let a student policy π_θ generate a completion $y = (y_1, \dots, y_T) \sim \pi_\theta(\cdot | x)$. We evaluate a rollout with K reward functions

$$\mathbf{r}(x, y) = (r_1(x, y), \dots, r_K(x, y)), \quad (1)$$

with priority ordering $r_1 \succ r_2 \succ \dots \succ r_K$. For a policy π , define performance on objective i as

$$J_i(\pi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(\cdot | x)}[r_i(x, y)]. \quad (2)$$

The ordering encodes an asymmetric preference: improvement on a lower-priority objective is not exchangeable for degradation of a higher-priority objective.

We are additionally given M frozen specialists $\{\pi_{\phi_1}, \dots, \pi_{\phi_M}\}$. Let $e(i) \in \{1, \dots, M\}$ map reward i to the expert used to correct that objective. The mapping need not be one-to-one: if one specialist is strong on several correlated rewards, multiple i may share the same $e(i)$.

3.2 ON-POLICY DISTILLATION

At the start of an update, we freeze an anchor $\bar{\pi}_\theta \leftarrow \pi_\theta$, sample student rollouts, and minimize a token-averaged reverse KL toward a fixed target distribution together with a hard-label cross entropy term (Agarwal et al., 2024; Hinton et al., 2015). The target is the routed specialist, the anchor itself, or the projected virtual teacher introduced below. Unlike domain-based MOPD (Ma et al., 2026), LMOPD chooses this target from the rollout’s reward profile. Appendix A.1 gives the complete objective and notation.

3.3 STAGE I: LEXICOGRAPHIC ROLLOUT ROUTING

We sample G rollouts for each prompt from the frozen anchor, as in group-based RL methods:

$$y_{b,g} \sim \bar{\pi}_\theta(\cdot | x_b), \quad b \in \{1, \dots, B\}, \quad g \in \{1, \dots, G\}. \quad (3)$$

Writing $r_{b,g,i} = r_i(x_b, y_{b,g})$, we compute the within-prompt moments and group-relative advantage:

$$\mu_{b,i} = \frac{1}{G} \sum_g r_{b,g,i}, \quad \sigma_{b,i} = \sqrt{\frac{1}{G} \sum_g (r_{b,g,i} - \mu_{b,i})^2}, \quad A_{b,g,i} = \frac{r_{b,g,i} - \mu_{b,i}}{\sigma_{b,i} + \epsilon}. \quad (4)$$

A single satisfaction rule is inappropriate for every reward type. For discrete rewards with a known maximum acceptable value $r_i^{\max}$, we use an absolute gate; for continuous rewards, we use relative underperformance:

$$g_i(x_b, y_{b,g}) = \mathbb{I}[r_{b,g,i} < r_i^{\max}] \text{ (discrete)}, \quad g_i(x_b, y_{b,g}) = \mathbb{I}[A_{b,g,i} < 0] \text{ (continuous)}. \quad (5)$$

The absolute gate is important for sparse binary rewards: if every rollout in a group is incorrect, all group-relative advantages are zero, yet every rollout still requires correctness supervision.

LMOPD selects the first objective whose gate fires:

$$k^*(x_b, y_{b,g}) = \min \{i \in \{1, \dots, K\} : g_i(x_b, y_{b,g}) = 1\}. \quad (6)$$

If the set is nonempty, the routed expert is $m^* = e(k^*)$. If every gate passes, no expert is selected and the rollout is self-distilled. Let $\beta_{m^*} \in (0, 1]$ be the routed-teacher coefficient used by the existing OPD logit pool; $\beta_{m^*} = 1$ recovers a direct specialist target. We use $\beta_{m^*} = 0.9$ for all experiments. Writing $p_{0,b,g,t} = \pi_\theta(\cdot \mid s_{b,g,t})$ and $p_{m,b,g,t} = \pi_{\phi_m}(\cdot \mid s_{b,g,t})$, the routing-only target is

$$q_{b,g,t}^{\text{route}} = \begin{cases} \text{softmax}((1 - \beta_{m^*}) \log p_{0,b,g,t} + \beta_{m^*} \log p_{m^*,b,g,t}), & \text{if at least one gate fires,} \\ p_{0,b,g,t}, & \text{otherwise.} \end{cases} \quad (7)$$

Substituting Eq. 7 into Eq. 13 gives the routing-only LMOPD ablation. The ordering therefore gives each objective a ‘‘first right of refusal’’ over the rollout: incorrect responses receive capability supervision before lower-priority properties such as conciseness are considered. For projection, routing also records the ordered set $\mathcal{C}(k^*)$ of distinct higher-priority experts, excluding the routed expert itself. Appendix A.2 gives the exact construction, edge cases, and pseudocode.

3.4 STAGE II: PRIORITY-AWARE FUNCTION-SPACE PROJECTION

Why routing alone is insufficient. Routing controls which expert supervises a rollout, but it does not control the direction of that expert’s update. A lower-priority expert can therefore assign a correction that locally opposes a higher-priority capability. LMOPD tests compatibility with an inner product between token-level corrections. The test concerns only the teacher-dependent reverse-KL component and excludes the hard-label cross-entropy term. This surrogate avoids an additional student backward pass per expert compared to computing the inner product on the actual gradients; more details can be found in Appendix A.3.

Centered log-policy corrections. For a token prefix s_t , write the frozen student anchor as $p_{0,t} = \pi_\theta(\cdot \mid s_t)$ and expert m as $p_{m,t} = \pi_{\phi_m}(\cdot \mid s_t)$. For every vocabulary item a , define

$$\Delta_{m,t}(a) = \log p_{m,t}(a) - \log p_{0,t}(a), \quad \tilde{\Delta}_{m,t}(a) = \Delta_{m,t}(a) - \mathbb{E}_{a' \sim p_{0,t}} [\Delta_{m,t}(a')]. \quad (8)$$

$\tilde{\Delta}_{m,t}(a)$ is just a centered version of $\Delta_{m,t}(a)$.

One-sided priority-aware projection. Suppose the rollout is routed to m^* and $\mathcal{C}(k^*) = (c_1, \dots, c_L)$. We initialize $d_t^{(0)} = \beta_{m^*} \tilde{\Delta}_{m^*,t}$, using the same routed-teacher coefficient as Eq. 7. We then visit the constraint experts once in priority order. Step ℓ is

$$u_{\ell,t} = \left\langle d_t^{(\ell-1)}, \tilde{\Delta}_{c_\ell,t} \right\rangle, \quad d_t^{(\ell)} = d_t^{(\ell-1)} - \frac{\min(u_{\ell,t}, 0)}{\|\tilde{\Delta}_{c_\ell,t}\|_2^2 + \epsilon} \tilde{\Delta}_{c_\ell,t}. \quad (9)$$

Each step removes only an opposing component, but a single sweep may leave conflicts among non-orthogonal constraints. This is a token-level surrogate, not a parameter-gradient or reward-level guarantee; Appendix A.3 gives the exact limiting cases. The projected virtual teacher is

$$q_{b,g,t}^{\text{proj}} = \text{softmax} \left(\log p_{0,t} + \tau d_t^{(L)} \right), \quad (10)$$

where the projection strength is $\tau = 1$ in our experiments. The complete method substitutes Eq. 10 into Eq. 13, with the target and projection coefficients detached from the student gradient. Appendix A.3 gives the complete algorithm, limiting cases, and one-sweep feasibility analysis. Appendix B describes the streamed Gram-space implementation and numerical safeguards.

4 EXPERIMENTS

4.1 TRAINING

Datasets. We use a proprietary mathematical-reasoning corpus for post-training.

Reward functions. We use binary answer correctness, reasoning quality judged by a 31B Gemma 4 judge (Team et al., 2026) as an average over four axes (reasoning correctness, verification, clarity, and efficiency), and a bounded length-based score S_{conc} . All rewards are in the interval $[0, 1]$. Every reported Conc metric is computed from raw S_{conc} . The conciseness specialist is trained with a correctness-gated version where it only receives S_{conc} on correct rollouts and 0 otherwise, whereas GDPO directly optimizes raw S_{conc} . Appendix C.5 gives the complete definitions.

The orders encode prerequisites rather than interchangeable utilities. Correctness comes first because presentation cannot redeem a wrong answer; reasoning correctness follows as a core capability, then verification, efficiency, and conciseness refine otherwise correct solutions. We therefore read results from highest to lowest priority and treat their macro-average as a descriptive summary.

Expert models. We train reward-specialized experts from the shared base and select teachers by target-reward validation performance. This produces the two integration settings below; See Appendix C.6 for further details on specialist training and expert selection.

Two-expert setting (three rewards).

The first setting asks whether a student can acquire a lower-priority behavior from one specialist while retaining the core mathematical capability of another. We order the objectives as $R_C \succ R_{\text{RQ}} \succ R_{\text{conc}}$ and use two teachers:

1. a capability expert, selected from the reasoning-specialized runs and used for both the correctness and aggregate reasoning-quality gates; and
2. a conciseness expert, used for the conciseness gate.

Because both higher-priority gates share one capability teacher, this setting isolates adding conciseness while retaining core mathematical capability. As we use one teacher for both R_C and R_{RQ} , rollouts that route to either one of those gates would get the same loss.

Four-expert setting (five rewards).

The second setting tests a longer priority chain with supervision distributed across more specialists. We use $R_C \succ R_{\text{RC}} \succ R_{\text{RV}} \succ R_{\text{RE}} \succ R_{\text{conc}}$. The four teachers are:

1. a reasoning-correctness expert, shared by the correctness and reasoning-correctness gates;
2. a reasoning-verification expert;
3. a reasoning-efficiency expert; and
4. a conciseness expert.

This setting tests integration under a longer chain in which distinct specialist corrections compete and later teachers act only after earlier gates pass. It therefore tests both multi-expert capability integration and the scaling behavior of the lexicographic routing rule.

Student model. The LMOPD student and all RL-trained specialist and baseline policies start from the same roughly 30B-parameter mixture-of-experts checkpoint, with approximately 3B active parameters per token. The checkpoint is an internal predecessor of Cohere’s North Mini Code (Team Cohere, 2026).

Baselines. We compare LMOPD with baselines that isolate different components of the method.

Rewarded Soup. Rewarded Soups (Ramé et al., 2023) linearly interpolates the parameters of reward-specialized policies trained from the same initialization. We evaluate both a uniform interpolation and a correctness-emphasized interpolation selected without using held-out test sets. This tests whether capability integration can be achieved by a static weight-space merge rather than by on-policy teacher supervision.

GDPO. GDPO (Liu et al., 2026a) is our primary multi-objective RL baseline. It optimizes raw correctness, aggregate reasoning quality, and raw S_{conc} , normalizing each reward dimension before combining the advantages into one policy-gradient direction. GDPO uses objective weights, rather

than hand-designed gates, to control the trade-off. We evaluate equal weights and correctness-emphasized 5:1 and 25:1 weightings; the main tables show equal weights and the best-tuned 25:1 run, while Table 4 retains the intermediate configuration.

Correctness-Gated RLVR. As a lexicographic reward-design control, Correctness-Gated RLVR makes correctness a prerequisite by zeroing the aggregate reasoning-quality and conciseness rewards on incorrect rollouts, then scalarizes the rewards that remain. Appendix C.7 gives both baseline objectives. Because R_{RQ} is the mean of the same four reasoning-quality axes, Correctness-Gated RLVR and each GDPO weighting receive the same three aggregate rewards in the two- and four-expert settings. We therefore reuse the same step-300 policy checkpoints in both tables; the four-expert table only decomposes aggregate reasoning quality into component axes for evaluation.

Individual specialists. We report the frozen reward-specialized experts themselves. These models are not unified multi-objective solutions; instead, they establish the per-objective headroom available from specialization and define the reference point for retained gain.

Component ablations. The routing-only ablation is exactly the same as LMOPD but skips the projection step. This isolates the contribution of priority-aware projection. We also present a random-routing control which is exactly the same as the routing-only variant but matches experts to rollouts at random. This tests whether improvements come merely from exposure to multiple teachers or from matching teacher identity to the rollout’s reward deficiencies and priority order.

Training settings. Two- and four-expert LMOPD variants train for 40 and 80 steps, respectively, and use their final checkpoints. All RL-trained specialist and baseline policies use their final step-300 checkpoint. See Appendix C for further details.

4.2 EVALUATION

Datasets and protocol. We generate 50 completions per problem on AIME 2024, AIME 2025, and HMMT February 2025 and report the average and standard error of the mean (SEM) None of these benchmarks is used for training or checkpoint selection. See Appendix C for further details.

Metrics and uncertainty. We aggregate each reported objective across the three benchmarks. We report pass@1, aggregate reasoning quality (plus the three relevant component axes), and raw S_{conc} independent of correctness. Raw objective values are means with standard errors of the mean (SEM), while retained-gain SEs use the delta method.

For objective i , let $J_i(\pi)$ denote the model’s mean performance, let π_0 be the shared base policy, and let π_i^* be the best available specialist for that objective. We report the *retained gain*

$$G_i(\pi) = \frac{J_i(\pi) - J_i(\pi_0)}{J_i(\pi_i^*) - J_i(\pi_0)} \times 100\%. \quad (11)$$

Retained gain measures how much of the improvement created by specialization survives after capabilities are integrated into one model. A value of 0% means that the unified method does not improve over the base on that objective, while 100% means that it closes the entire base-to-specialist gap; values above 100% indicate that the integrated policy surpasses the best specialist, and negative values indicate degradation relative to the base. This normalization is useful because reward axes can differ substantially in both scale and available headroom. We report the corresponding raw objective values in Table 4 and treat the macro-average retained gain as a descriptive summary rather than a substitute for inspecting high-priority objectives individually.

5 RESULTS

5.1 TWO-EXPERT RETAINED GAIN

Table 1 reports retained gain aggregated over AIME 2024, AIME 2025, and HMMT February 2025, with 50 sampled completions per problem. LMOPD preserves the two high-priority capabilities: its point estimates retain $(102.9 \pm 15.5)\%$ of the accuracy gain (raw: $0.711 \rightarrow 0.745$) and $(103.3 \pm 5.7)\%$ of the aggregate reasoning-quality gain (raw: $0.719 \rightarrow 0.772$), while retaining $(46.9 \pm 2.2)\%$ of the

Table 1: Two-expert retained gain. LMOPD has the highest average among unified methods while retaining gains in accuracy and aggregate reasoning quality.

Model	pass@1	RQ	Conc	Average
<i>Weight-space merging</i>				
Rewarded Soup (uniform; 1:1)	61.2 $\pm$ 13.7%	61.6 $\pm$ 5.0%	67.2 $\pm$ 2.3%	63.3%
Rewarded Soup (correctness-emphasized; 2:1)	<u>72.7 $\pm$ 14.0%</u>	<u>84.3 $\pm$ 5.3%</u>	44.4 $\pm$ 2.1%	<u>67.1%</u>
<i>Multi-reward reinforcement learning</i>				
Correctness-Gated RLVR (1,1,1)	70.0 $\pm$ 13.8%	40.2 $\pm$ 5.0%	49.1 $\pm$ 2.2%	53.1%
GDPO (1,1,1)	-457.9 $\pm$ 85.2%	-154.4 $\pm$ 15.3%	355.8 $\pm$ 9.8%	-85.5%
GDPO (25,1,1)	-31.5 $\pm$ 20.1%	-2.4 $\pm$ 4.7%	<u>90.5 $\pm$ 3.7%</u>	18.9%
<i>Ours</i>				
LMOPD	102.9 $\pm$ 15.5%	103.3 $\pm$ 5.7%	46.9 $\pm$ 2.2%	84.4%
<i>Reference policies (not unified)</i>				
Base	0%	0%	0%	0%
Reasoning Quality + Correctness Expert	100%	100%	39.2 $\pm$ 2.2%	79.7%
Conciseness Expert	0.8 $\pm$ 16.1%	-20.4 $\pm$ 6.5%	100%	26.8%

Bold and underlined entries are the best and second-best unified methods in each column; reference policies are excluded. LMOPD denotes routing plus priority-aware projection and uses the fixed step-40 checkpoint. Best specialist per axis: pass@1 = Reasoning Quality + Correctness Expert (0.7440), RQ = Reasoning Quality + Correctness Expert (0.7698), and Conc = Conciseness Expert (0.8131). The shared base defines 0%.

conciseness gain. Because 100% denotes the best-specialist endpoint, both high-priority estimates are consistent within their SEs with specialist-level performance.

LMOPD has the highest performance among unified methods (84.4%), 17.3 points above the strongest non-LMOPD baseline, the correctness-emphasized Rewarded Soup (67.1%). That soup retains substantial gains in aggregate reasoning quality and conciseness but only 72.7% of the available accuracy gain. Equal-weight GDPO reduces mean completion length from 16.3k tokens for the base to 6.2k while lowering raw accuracy from 0.711 to 0.559. At 25:1, GDPO avoids this collapse: mean length rises to 13.3k tokens, and raw accuracy and aggregate reasoning quality recover to 0.700 and 0.717, respectively, while retaining 90.5% of the conciseness gain. The equal-weight run is therefore consistent with length-dominant reward hacking, while stronger correctness weighting substantially mitigates it.

For qualitative analysis of LMOPD’s completions, we present a case study of an individual eval problem in Appendix D.

5.2 FOUR-EXPERT RETAINED GAIN

The four-expert setting tests whether high-priority capabilities remain protected as the priority chain grows to five objectives. Table 2 shows that LMOPD retains (90.6 $\pm$ 15.0)% of the accuracy gain (raw: 0.711 $\rightarrow$ 0.748) and (88.6 $\pm$ 7.0)% of the reasoning-correctness gain (raw: 0.755 $\rightarrow$ 0.805). Every other unified method reaches at most 57.1% and 63.5%, respectively. LMOPD also has the highest macro-average point estimate (39.6%, versus 34.4% for Correctness-Gated RLVR).

Because the objectives are lexicographically ordered, the columns should be read in priority order rather than as equally consequential targets. The central scaling result is therefore LMOPD’s large margin on accuracy and reasoning correctness, followed by a positive composite-conciseness gain (28.0 $\pm$ 1.1%). Reasoning verification and efficiency are lower-priority objectives: their retained gains of -6.8% and -2.4% do not outweigh preservation of the first two objectives under the stated preference order. This profile is consistent with the intended behavior of LMOPD—preserving high-priority capabilities even when all specialist directions cannot be improved simultaneously. **LMOPD outperforms other multi-objective optimization baselines on important objectives.**

5.3 COMPONENT ABLATIONS

Table 3 separates teacher assignment from priority-aware projection. Lexicographic routing’s average retained-gain exceeds random routing by 28.7 points with two experts and 23.0 points with four. In the four-expert setting, random routing improves later objectives but falls below the base on accuracy and reasoning correctness; teacher exposure alone is therefore insufficient.

Table 2: Four-expert retained gain, with objectives ordered by priority. LMOPD leads unified methods on composite pass@1, RQ-corr, and average.

Model	Composite pass@1	RQ-corr	RQ-verif	RQ-eff	Composite Conc	Average
<i>Weight-space merging</i>						
Rewarded Soup (uniform; 1:1:1:1)	31.4 ± 14.3%	51.6 ± 7.5%	-2.3 ± 2.5%	28.7 ± 2.1%	40.2 ± 1.4%	29.9%
Rewarded Soup (correctness-emphasized; 2:1:1:1)	30.5 ± 11.6%	63.5 ± 10.4%	4.2 ± 2.9%	19.2 ± 2.0%	28.5 ± 0.8%	29.2%
<i>Multi-reward reinforcement learning</i>						
Correctness-Gated RLVR (1,1,1)	57.1 ± 9.5%	55.7 ± 7.7%	3.2 ± 2.4%	30.8 ± 4.4%	25.2 ± 1.8%	34.4%
GDPO (1,1,1)	-373.4 ± 55.7%	-265.0 ± 35.7%	-161.5 ± 6.1%	95.1 ± 2.0%	182.1 ± 3.0%	-104.6%
GDPO (25,1,1)	-25.7 ± 14.0%	5.6 ± 9.1%	-35.6 ± 3.6%	<u>36.0 ± 2.4%</u>	<u>46.3 ± 1.9%</u>	5.3%
<i>Ours</i>						
LMOPD	90.6 ± 15.0%	88.6 ± 7.0%	-6.8 ± 2.7%	-2.4 ± 2.8%	28.0 ± 1.1%	39.6%
<i>Reference policies (not unified)</i>						
Base	0%	0%	0%	0%	0%	0%
RQ-Correctness Expert	100%	100%	7.7 ± 3.2%	-8.9 ± 3.2%	13.0 ± 0.9%	42.4%
RQ-Verification Expert	79.7 ± 12.5%	124.4 ± 9.5%	100%	-58.5 ± 4.4%	-44.3 ± 2.1%	40.3%
RQ-Efficiency Expert	-202.8 ± 33.7%	-118.5 ± 20.7%	-106.5 ± 4.9%	100%	100%	-45.6%
Conciseness Expert	0.7 ± 12.8%	25.3 ± 9.9%	-47.6 ± 3.3%	21.7 ± 2.1%	51.2 ± 1.2%	10.2%

Bold and underlined entries are the best and second-best unified methods in each column; reference policies are excluded. LMOPD denotes routing plus priority-aware projection. Best specialist per axis: pass@1 = RQ-Correctness Expert (0.7515), RQ-corr = RQ-Correctness Expert (0.8117), RQ-verif = RQ-Verification Expert (0.7902), RQ-eff = RQ-Efficiency Expert (0.7827), and Conc = RQ-Efficiency Expert (0.8402).

Table 3: Routing and projection ablations. Lexicographic routing has higher performance than random routing; matched four-expert projection shifts retention toward the top priorities.

(a) Two experts						
Variant	pass@1	RQ	Conc	Average		
LMOPD	102.9 ± 15.5%	103.3 ± 5.7%	46.9 ± 2.2%	84.4%		
LMOPD (routing only)	95.3 ± 12.5%	97.3 ± 4.8%	43.5 ± 1.8%	78.7%		
LMOPD (random routing)	30.4 ± 14.0%	46.8 ± 4.2%	72.8 ± 2.0%	50.0%		
(b) Four experts						
Variant	Composite pass@1	RQ-corr	RQ-verif	RQ-eff	Composite Conc	Average
LMOPD	90.6 ± 15.0%	88.6 ± 7.0%	−6.8 ± 2.7%	−2.4 ± 2.8%	28.0 ± 1.1%	39.6%
LMOPD (routing only)	54.0 ± 13.2%	69.5 ± 11.1%	−18.4 ± 2.9%	4.3 ± 3.4%	39.7 ± 1.0%	29.8%
LMOPD (random routing)	−53.7 ± 14.2%	−2.0 ± 13.6%	−39.6 ± 3.6%	53.8 ± 2.7%	75.6 ± 1.6%	6.8%

Projection exposure differs sharply by setting. In the two-expert run, only 0.6% of rows carry a higher-priority constraint, corresponding to 0.003–0.005 constraints per token and a projected-to-original norm ratio of 0.9989. Projection is therefore effectively inert there, so the performance shifts from routing only to full LMOPD are modest relative to their uncertainty. In the four-expert run, projection is materially engaged: it processes 0.26 constraints per token, adds constraint forwards for 13.7% of rows, and yields a norm ratio of 0.91. At the matched step-80 checkpoint, projection shifts the retained-gain point estimates for accuracy and reasoning correctness from 54.0% to 90.6% and from 69.5% to 88.6%, respectively, while reducing later-axis gains.

6 LIMITATIONS

LMOPD serves as a local heuristic rather than global lexicographic optimality: its function-space test approximates parameter-gradient compatibility, and one sweep may leave residual conflicts.

Strict routing can starve later experts (Appendix A.4), and outcomes ultimately depend on reward and teacher quality. We evaluate only proprietary-model mathematical reasoning; open-weight and cross-domain validation remain future work. Because 30B-scale post-training is costly, multi-seed replication is limited to two-expert LMOPD: three 40 step runs (see Appendix C.4). The main-table run is the median seed by average retained gain (84.4%; the other seeds obtain 81.2% and 95.1%); Table 5 and fig. 2 give the complete results. Except for this replication study, reported SEs quantify evaluation sampling, not training-run variation.

7 CONCLUSION

We introduced LMOPD, which routes each rollout to the specialist for its highest-priority unmet objective and projects that specialist’s token-level correction against earlier experts. With two experts, LMOPD retains specialist-level accuracy and aggregate reasoning quality while acquiring conciseness; with four, it retains 90.6% of the accuracy gain and 88.6% of the gain in reasoning correctness, with higher point estimates than multi-objective baselines on both top priorities. Ablations show that deficiency-aware routing is essential and that, when materially engaged with four experts, projection shifts point estimates toward these priorities. Together, these results position rollout-conditioned distillation as a practical path to integrating specialists without collapsing prioritized capabilities into a single global trade-off.

AI USE STATEMENT

In this work, we used generative AI tools for coding assistance in implementing methods, designing and providing feedback on methods and experiments, refining hypotheses, and supporting qualitative data analysis. We have not used generative AI tools for interpreting results, cleaning and reformatting datasets, and generating synthetic datasets, developing theoretical models or frameworks, formulating mathematical claims, providing critical ingredients for proving mathematical claims, writing of proofs, translation are not applicable to this work. Additionally, we used generative AI tools for modifying scientific figures, editing the research paper, analyzing existing literature. We have reviewed all AI-assisted work. LLM-generated code, experiment feedback, and paper modifications were verified by the main author for correctness. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REPRODUCIBILITY STATEMENT

The LMOPD routing rule is given in Eqs. 4–7 and Algorithm 1; the priority-aware target is specified in Eqs. 8–10, Algorithm 2, and the Gram-space implementation in Appendix B; and the distillation objective is specified in Eqs. 13–15. The experimental setup, baselines, and retained-gain metric are summarized in Section 4, with reward definitions, training schedules, batch construction, and evaluation details in Appendix C. The base checkpoint and training corpus used in our experiments are proprietary, so those exact assets cannot be released. The method itself requires only a student policy, white-box access to reward-specialized teacher distributions, and the stated reward functions, and can therefore be reproduced with open-weight models by training specialists from a common initialization. The paper provides implementation-level pseudocode and configuration details sufficient to reproduce the algorithm independently of the proprietary checkpoint.

REFERENCES

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/5be69a584901a26c521c2b51e40a4c20-Abstract-Conference.html.
- Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025. URL <https://arxiv.org/abs/2505.23281>.
- Hunar Batra, Haoqin Tu, Hardy Chen, Yuanze Lin, Cihang Xie, and Ronald Clark. Spatial-Thinker: Reinforcing 3d reasoning in multimodal LLMs via spatial rewards, 2025. URL <https://arxiv.org/abs/2511.07403>.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/8ac015d409635f196f9e3e9dcfb9a94e-Abstract-Conference.html.

- Harvard-MIT Mathematics Tournament. February 2025 tournament archive. HMMT Archive, 2025. URL <https://www.hmmt.org/www/archive/282>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. GDPO: Group reward-decoupled normalization policy optimization for multi-reward RL optimization, 2026a. URL <https://arxiv.org/abs/2601.05242>.
- Xiaojuan Liu, Han Xu, Jianqiang Xia, Zhixuan Li, Ke Xu, Yiwei Dai, Xinran Chen, Changwo Wu, and Yuchen Li. SCOPE-RL: Optimizing reasoning paths before and after success, 2026b. URL <https://arxiv.org/abs/2607.11506>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. MOPD: Multi-teacher on-policy distillation for capability integration in LLM post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- Alexandre Ramé, Guillaume Couairon, Mustafa Shukor, Corentin Dancette, Jean-Baptiste Gaya, Laure Soulier, and Matthieu Cord. Rewarded soups: Towards pareto-optimal alignment by interpolating weights fine-tuned on diverse rewards. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/e12a3b98b67e8395f639fde4c2b03168-Abstract-Conference.html.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, Mayank Chaturvedi, Aditya Chawla, Victor Cotruta, Alice Coucke, Phil Culliton, Robert Dadashi, Lucas Dixon, Mohamed Elhawaty, Utku Evci, Clément Farabet, Johan Ferret, Filippo Galgani, Sertan Girgin, Jean-Bastien Grill, Maarten Grootendorst, Jiaxian Guo, Cassidy Hardin, Yanzhang He, Steven M. Hernandez, Omri Homburger, Léonard Hussenot, Juyeong Ji, Armand Joulin, Aishwarya Kamath, Parnian Kassraie, Olivier Lacombe, Preethi Lahoti, Gaël Liu, Gus Martins, Luciano Martins, Tatiana Matejovicova, Ramona Merhej, Nikola Momchev, Sneha Mondal, Ryan Mullins, Sindhu Raghuram Panyam, Shreya Pathak, Sarah Perrin, André Susano Pinto, Etienne Pot, Angéline Pouget, Alexandre Ramé, Sabela Ramos, Douglas Reid, David Rim, Morgane Rivière, Karsten Roth, Louis Rouillard, Omar Sanseviero, Pier Giuseppe Sessa, Shane Settle, Danila Sinopalnikov, Sara Smoot, Piotr Stanczyk, Andreas Steiner, Lawrence Stewart, Ilya Tolstikhin, Michael Tschannen, Anton Tsitsulin, Nino Vieillard, Renjie Wu, Pingmei Xu, Haichuan Yang, Edouard Yvinec, Biao Zhang, Li Zhang, Joe Zou, Nicolas Aagnes, Abdelrahman Abdelhamed, Jakub Adamek, Shivani Agrawal, Shubham Agrawal, Ibrahim Alabdulmohsin, Jean Baptiste Alayrac, Uri Alon, Chandramouli Amarnath, Ankesh Anand, Chrysovalantis Anastasiou, Setareh Ariafar, François-Xavier Aubet, Kyriakos Axiotis, Federico Barbero, Joelle Barral, Alexei Bendebury, Urs Bergmann, Stanley Bileschi, Kat Black, Mathieu Blondel, Sebastian Borgeaud, Arthur Bražinskas, Ryan Burrell, Robert Busa-Fekete, Mu Cai, Daniele Calandriello, Glenn Cameron, Charlotte Caucheteux, Rahma Chaabouni, Garima Chadha, Jetha Chan, Blake Jianhang Chen, Jesse Chen, Lin Chen, Xu Chen, Derek Cheng, Tzu hsiang Chien, Nikolai Chinaev, Yi Chou, Zhaohui Chu, Benjamin Coleman, Pooja Consul, Sam Conway-Rahman, Scott Crowell, Dylan Cutler, Vivek Dani, Samira Daruki, Anil Das, Daniel Deutsch, Nishanth Dikkala, Li Ding, Qiuhan Ding, Shenil Dodhia, Konstantin Donhauser, Tulsee Doshi, Anca Dragan, Alex Druinsky, Sahil Dua, Zoltan Egyed, Danielle Eisenbud, Daniel Eppens, Cindy Fan, Bahare Fatemi, Yassir Fathullah, Vlad Feinberg,

- Milen Ferev, Sebastian Flennerhag, Takumi Fujimoto, João Gabriel Oliveira, Isaac Galatzer-Levy, João Gante, Simon Geisler, Soham Ghosal, Antonious M. Girgis, Tamara von Glehn, Alec Go, Alhaad Gokhale, Alex Grills, Yiming Gu, Mayank Gupta, Pramod Gupta, Guru Guruganesh, Raia Hadsell, Hamza Harkous, Jitendra Harlalka, Demis Hassabis, Anja Hauth, Joe Heyward, Arian Hosseini, Chih-Yang Hsia, I-Hung Hsu, Xiaopeng Huang, Yangsibo Huang, Kevin Hui, Adrian Hutter, Te I, Fotis Iliopoulos, Advait Jain, Ganesh Jawahar, Ziwei Ji, Qilin Jin, Melvin Johnson, Kandarp Joshi, Arun Kandoor, Wang-Cheng Kang, Koray Kavukcuoglu, Mehran Kazemi, Kathleen Kenealy, Amr Khalifa, Phoebe Kirk, Ivan Korotkov, Suraj Kothawade, Vitaly Kovalev, Neel Kovelamudi, Adam Kraft, Ravin Kumar, Vivek Kumar, Harish Kuppam, Justin Lannin, Chen-Yu Lee, Seungji Lee, Dmitry Lepikhin, Alon Levkovitch, Dongdong Li, Qijia Li, Valentin Liévin, Ethan Lin, Ziqian Lin, Casper Liu, Tianlin Liu, Tianqi Liu, Xin Liu, Ivan Lobov, Mayank Lunayach, Min Ma, Gagan Madan, Andrii Maksai, Eric Malmi, Michal Matuszak, Daniel McDuff, Gaurav Menghani, Maciej Mikuła, Daniil Mirylenka, Karolis Misiunas, Vedant Misra, Andreea Mitran, Kareem Mohamed, Maksim Mukha, Eric Noland, James O'Donnell, Brendan O'Donoghue, Kate Olszewska, Bernett Orlando, Wanqiong Pan, Rina Panigrahy, Unnati Parekh, Nicolas Perez-Nieves, Chunjong Park, Eric Paskie, Liqian Peng, Bryce Petrini, Slav Petrov, Jonas Pfeiffer, Bilal Piot, Martyna Plomecka, Siim Poder, Octavio Ponce, Arijit Pramanik, David Racz, Anish Rajan, Michelle Ramanovich, Anand Rao, Marvin Ritter, Vitor Rodrigues, Evan Rosen, Mikołaj Rybiński, Noveen Sachdeva, Michaël E. Sander, Rohit Sathyanarayana, Sagar Savla, Samuel Schmidgall, Tal Schuster, George Scrivener, Benoit Seguin, Andrew Sellergren, Aliaksei Severyn, Izhak Shafran, Dhruv Shah, Bobak Shahriari, Yuan Shang-guan, Ashish Shenoy, Pradeep Shenoy, Rakesh Shivanna, Pauline Sho, Lucas Spangher, Wojciech Stokowiec, Tim Strother, Yao Su, Yinghao Sun, Mukund Sundararajan, Andrea Tacchetti, Mor Hazan Taege, Pouya Tafti, Jean Tarbouriech, Chetan Tekur, Shantanu Thakoor, Rahul Thapa, Madeleine Traverse, Lenart Treven, Tao Tu, Chien Te Tung, Çağlar Ünlü, Petar Veličković, Malini Pooni Venkat, Sagar Gubbi Venkatesh, Vidya Venkiteswaran, Francesco Visin, Alex Vitvitskyi, Kiran Vodrahalli, Weiye Wang, Xin Wang, Tris Warkentin, Jan Wassenberg, John Wieting, Cindy Wu, Lechao Xiao, Hao Xu, Yuhui Xu, Fuzhao Xue, Arun Yadav, Jun Yan, Antoine Yang, Lin Yang, Ming-Hsuan Yang, Ziyu Ying, Jae Hyeon Yoo, Morteza Zadimoghaddam, Sajjad Zafar, Fred Zhang, Jiageng Zhang, Jianyi Zhang, Xiaofan Zhang, Chao Zhao, David Zhou, and Chen Zou. Gemma 4 technical report, 2026. URL <https://arxiv.org/abs/2607.02770>.
- Team Cohere. Introducing North Mini Code: Cohere’s first model for developers, June 2026. URL cohere.com/blog/north-mini-code.
- Alperen Tercan and Vinayak S. Prabhu. Thresholded lexicographic ordered multiobjective reinforcement learning. In *ECAI 2024*, pp. 3006–3014, 2024. doi: 10.3233/FAIA240841.
- Wen Wang, Jiahua Bao, Yongsiqu Tu, Yihao Liu, Haotian Zhou, Haoxuan Ma, Mengyu Zhou, Wenkui Fan, Junwei He, Xiaoxi Jiang, and Guanjin Jiang. SMOPD: Multi-reward reinforcement learning via specialize-and-merge online policy distillation, 2026. URL <https://arxiv.org/abs/2608.03092>.
- Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.
- Bo Xue, Dake Bu, Ji Cheng, Yuanyu Wan, and Qingfu Zhang. Multi-objective linear reinforcement learning with lexicographic rewards. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 70065–70085. PMLR, 2025. URL <https://proceedings.mlr.press/v267/xue25a.html>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui

Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, et al. Nemotron-cascade 2: Post-training llms with cascade rl and multi-domain on-policy distillation. *arXiv preprint arXiv:2603.19220*, 2026.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 33, pp. 5824–5836, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/3fe78a8acf5fda99de95303940a2420c-Abstract.html>.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

A ADDITIONAL METHOD DETAILS

A.1 ON-POLICY DISTILLATION OBJECTIVE

At the start of an update, we freeze an anchor $\bar{\pi}_\theta \leftarrow \pi_\theta$ and sample rollouts from this snapshot. On-policy distillation evaluates its target on prefixes produced by the student rather than on a fixed teacher-generated corpus (Agarwal et al., 2024). For rollout n , define

$$s_t^{(n)} = (x^{(n)}, y_{<t}^{(n)}), \quad y^{(n)} \sim \bar{\pi}_\theta(\cdot \mid x^{(n)}). \quad (12)$$

Let $q_t^{(n)}(\cdot \mid s_t^{(n)})$ be a fixed token-level target distribution. Depending on the method variant, it is either the routed specialist, the anchor itself, or the projected virtual teacher. We use a token-averaged reverse-KL distillation loss

$$\mathcal{L}_{\text{KD}}^{(n)} = \frac{1}{T_n} \sum_{t=1}^{T_n} D_{\text{KL}}\left(\pi_\theta(\cdot \mid s_t^{(n)}) \parallel q_t^{(n)}(\cdot \mid s_t^{(n)})\right). \quad (13)$$

Following our training implementation, we add a regularizing hard-label cross-entropy term on the student-generated trajectory,

$$\mathcal{L}_{\text{hard}}^{(n)} = -\frac{1}{T_n} \sum_{t=1}^{T_n} \log \pi_\theta\left(y_t^{(n)} \mid s_t^{(n)}\right), \quad (14)$$

and optimize

$$\mathcal{L}_{\text{OPD}} = \frac{1}{N} \sum_{n=1}^N \left(\mathcal{L}_{\text{KD}}^{(n)} + \beta_{\text{hard}} \mathcal{L}_{\text{hard}}^{(n)} \right), \quad \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{OPD}}. \quad (15)$$

We use $\beta_{\text{hard}} = 1 - \beta_{\text{m*}} = 0.1$ for all experiments. Reverse KL is well suited to mode-seeking generative distillation (Gu et al., 2024); the hard-label cross-entropy term regularizes the update toward the sampled on-policy behavior.

A.2 ROUTING CONSTRAINT CONSTRUCTION

Routing intuition. The priority ordering gives each objective a “first right of refusal” over the rollout. With correctness first, an incorrect response is always corrected by the capability expert and cannot be pushed toward conciseness merely because it is short. Once correctness passes, the next objective is allowed to inspect the rollout, and so on. Continuous objectives use group-relative advantage so that the method asks whether a rollout is unusually weak on that dimension compared with alternative completions of the same prompt. Self-distillation prevents trajectories that already satisfy every gate from being arbitrarily pulled toward a specialist. The resulting update is still dense at the token level; only the identity of the teacher is selected lexicographically.

Higher-priority constraint list. Routing records the ordered list of distinct experts associated with objectives above k^* ,

$$\mathcal{C}(k^*) = \text{ordered_unique}(e(1), \dots, e(k^* - 1)) \setminus \{m^*\}. \quad (16)$$

The order is the objective-priority order, not checkpoint order. Removing m^* is important when one expert serves multiple objectives: an expert is never projected against itself, while other distinct experts attached to earlier objectives remain constraints, including experts that occur between two objectives served by m^* .

Algorithm 1 Lexicographic rollout routing

Require: Rollout $(x_b, y_{b,g})$; gates $g_1, \dots, g_K$; reward-to-expert map e

```

1:  $\mathcal{C} \leftarrow ()$  ▷ ordered list of passed experts
2: for  $i = 1, \dots, K$  do
3:   if  $g_i(x_b, y_{b,g}) = 1$  then
4:      $m^* \leftarrow e(i)$ 
5:     Remove  $m^*$  from  $\mathcal{C}$  if present
6:     return  $(m^*, \mathcal{C})$ 
7:   end if
8:   if  $e(i) \notin \mathcal{C}$  then
9:     Append  $e(i)$  to  $\mathcal{C}$ 
10:  end if
11: end for
12: return  $(\emptyset, ())$  ▷ all gates pass: self-distill

```

A.3 PROJECTION DERIVATION AND ALGORITHM

Parameter-gradient motivation. Let $z_{\theta,t}$ be the student logits at prefix s_t , $p_{\theta,t} = \text{softmax}(z_{\theta,t})$, and $J_t = \partial z_{\theta,t} / \partial \theta$. At the frozen anchor, define the categorical Fisher matrix

$$F_{0,t} = \text{diag}(p_{0,t}) - p_{0,t} p_{0,t}^\top. \quad (17)$$

For a detached virtual-teacher target $q_{m,t} = \text{softmax}(\log p_{0,t} + \tau d_{m,t})$, the teacher-dependent reverse-KL descent direction at the anchor is

$$g_{m,t}^{\text{KD}} := -\nabla_{\theta} D_{\text{KL}}(p_{\theta,t} \| q_{m,t})|_{\theta=\bar{\theta}} = \tau J_t^\top F_{0,t} d_{m,t}. \quad (18)$$

Consequently, exact token-local compatibility between two such KD components is

$$\langle g_{m,t}^{\text{KD}}, g_{j,t}^{\text{KD}} \rangle = \tau^2 d_{m,t}^\top M_t d_{j,t}, \quad M_t = F_{0,t} J_t J_t^\top F_{0,t}. \quad (19)$$

Centering does not remove the Fisher factor: for a $p_{0,t}$ -centered direction, $F_{0,t} d_{m,t} = \text{diag}(p_{0,t}) d_{m,t}$. A full trajectory-gradient comparison additionally contains cross-token blocks $F_{0,t} J_t J_u^\top F_{0,u}$. The hard-label cross-entropy term is not included here because it is independent of teacher identity; LMOPD modifies only the teacher-dependent KD component.

Computing the parameter directions requires an expert-specific vector–Jacobian product. LMOPD instead tests $d_{m,t}^\top d_{j,t}$, thereby replacing the complete pullback metric M_t with the identity and enforcing compatibility separately at each token. It is a heuristic on centered log-policy displacements and does not guarantee parameter-gradient or reward-level compatibility. Because its vocabulary inner product is unweighted, low-probability and high-probability actions have equal geometric weight even though the exact reverse-KL direction is Fisher-weighted.

Limiting cases. When $\mathcal{C}$ is empty and $\tau = 1$, Eq. 10 is exactly the routing-only logit pool in Eq. 7; when $\beta_{m^*} = 1$, both reduce to the routed expert distribution. Centering does not change this equality because it adds only an action-independent constant. Rows for which every gate passes retain all-zero expert coefficients, remain equal to $p_{0,t}$, and never enter the projection. Consequently, disabling projection does not change rollout selection or the unprojected target.

Algorithm 2 Priority-aware target for one routed rollout

Require: Frozen anchor p_0 ; routed expert m^* ; ordered constraints $\mathcal{C} = (c_1, \dots, c_L)$; coefficients

```

 $\beta_{m^*}, \tau$ 
1: if  $m^* = \emptyset$  then
2:   return  $q_t \leftarrow p_{0,t}$  for every token  $t$ 
3: end if
4: for completion token  $t = 1, \dots, T$  do
5:   Compute centered deltas  $\tilde{\Delta}_{m^*,t}$  and  $\{\tilde{\Delta}_{c_\ell,t}\}_{\ell=1}^L$  using Eq. 8
6:    $d_t \leftarrow \beta_{m^*} \tilde{\Delta}_{m^*,t}$ 
7:   for  $\ell = 1, \dots, L$  do
8:      $u \leftarrow \langle d_t, \tilde{\Delta}_{c_\ell,t} \rangle$ 
9:     if  $u < 0$  then
10:       $d_t \leftarrow d_t - \frac{u}{\|\tilde{\Delta}_{c_\ell,t}\|_2^2 + \epsilon} \tilde{\Delta}_{c_\ell,t}$ 
11:     end if
12:   end for
13:    $q_t \leftarrow \text{softmax}(\log p_{0,t} + \tau d_t)$ 
14: end for
15: return  $\{q_t\}_{t=1}^T$ 

```

One sweep versus exact feasibility. With at most one active constraint, the update is the exact orthogonal projection onto the corresponding half-space, up to ϵ and numerical guards. With two or more non-orthogonal constraints, a later step can reintroduce a negative inner product with an earlier expert. Repeated sweeps converge toward the intersection of the half-spaces, but our implementation uses one sweep by default because most routed rows have zero or one constraint.

Efficient implementation. Because the projected direction remains in the span of the expert corrections, we implement the same function-space sweep in coefficient space using streamed per-token $M \times M$ Gram matrices. This avoids both vocabulary-sized correction tensors and additional student backward passes. Appendix B gives the algebraic construction, streaming procedure, and numerical safeguards.

A.4 ROUTING STARVATION

An expert at position i is selected only when all higher-priority gates pass and its own gate fails. Abstractly,

$$\Pr[z = e(i)] = \Pr[g_1 = 0, \dots, g_{i-1} = 0, g_i = 1]. \quad (20)$$

As the number of priorities grows, later experts may receive fewer rollouts, especially early in training. This is an inherent consequence of strict routing and suggests extensions such as minimum routing quotas, softened priority constraints, or adaptive gate thresholds when scaling to many objectives.

B GRAM-SPACE PROJECTION IMPLEMENTATION

The dense presentation in Section 3.4 would require a [batch, sequence, vocabulary] correction tensor. The production implementation instead observes that every projected direction remains in the span of the centered expert deltas. Define the per-token Gram matrix

$$G_{t,mj} = \langle \tilde{\Delta}_{m,t}, \tilde{\Delta}_{j,t} \rangle. \quad (21)$$

If $d_t = \sum_m a_{m,t} \tilde{\Delta}_{m,t}$, then

$$\|d_t\|_2^2 = a_t^\top G_t a_t, \quad \langle d_t, \tilde{\Delta}_{j,t} \rangle = (a_t^\top G_t)_j. \quad (22)$$

A projection against expert j therefore changes only its coefficient,

$$a_{j,t} \leftarrow a_{j,t} - \text{clip}\left(\frac{(a_t^\top G_t)_j}{G_{t,jj} + \epsilon}, -c_{\max}, c_{\max}\right), \quad (23)$$

whenever the numerator is negative and the norm guards pass.

Because d_t stays in this span, the protected target remains a linear logit pool:

$$\log q_t^{\text{proj}} \stackrel{+C}{=} \left(1 - \tau \sum_m a_{m,t}\right) \log p_{0,t} + \tau \sum_m a_{m,t} \log p_{m,t}, \quad (24)$$

where $\stackrel{+C}{=}$ denotes equality up to an action-independent constant. The existing OPD kernel can therefore reconstruct the target from per-token pool coefficients rather than materializing the dense direction.

We compute G_t in two streamed passes over vocabulary blocks: one pass for the student and expert log-normalizers and one for the raw first and second moments needed to form the centered Gram matrix. The projection sweep itself is $O(M^2)$ per token after this computation and stores only the $M \times M$ Gram matrix, rather than a vocabulary-sized delta. Selective teacher forwarding is expanded from the routed expert to the union of the routed and higher-priority constraint experts for each row; lower-priority experts are never forwarded.

Numerical safeguards. Before the sweep, the implementation normalizes a_t by $\sqrt{a_t^\top G_t a_t}$ and restores its original scale afterward, making guard behavior invariant to the routed coefficient’s magnitude. Reported runs use centered deltas, $\epsilon = 10^{-8}$, an absolute constraint-norm floor of 10^{-10} , a relative floor $G_{t,jj} \geq 10^{-6} \|d_t\|_2^2$, $c_{\max} = 10$, and a final direction-preserving rescaling when $\|\tau a_t\|_1 > 20$. These guards bound unstable coefficient representations: the per-step guards skip or cap unsafe corrections, while the final rescaling preserves the resulting direction.

C EXPERIMENTAL DETAILS

C.1 TRAINING PROTOCOL

We study LMOPD on mathematical reasoning using a proprietary internal training set. All policies start from the same post-SFT checkpoint: a roughly 30B-parameter mixture-of-experts model with approximately 3B active parameters per token. The checkpoint is an internal predecessor of Cohere’s North Mini Code (Team Cohere, 2026). The checkpoint has completed general pretraining and supervised post-training on multiple domains, including mathematical reasoning, code, and instruction following. We train and evaluate with a maximum context length of 32K tokens.

Because LMOPD distillation dynamics differ from RLVR and with the number of experts to be distilled from, we run 40 training steps for all 2 expert LMOPD variants, and 80 steps for 4 expert LMOPD variants. Each step uses 64 prompts with 8 freshly sampled rollouts per prompt, giving 512 fully on-policy trajectories per update. We only filter out empty responses. We use a two-stage learning-rate schedule: 4×10^{-6} for the first 40 steps and 4×10^{-7} thereafter. All reward-specialist and direct multi-reward RL runs use a common 300-step horizon and their final step-300 checkpoints, with additional filters for zero-variance groups.

C.2 EVALUATION PROTOCOL

Our final evaluation suite contains AIME 2024, AIME 2025, and HMMT February 2025. AIME is a standard short-answer competition benchmark used extensively in recent LLM reasoning reports, including Qwen3 (Yang et al., 2025). HMMT February 2025 provides a distinct set of recent competition problems from the Harvard–MIT Mathematics Tournament (Harvard-MIT Mathematics Tournament, 2025) and is included in the MathArena evaluation ecosystem (Balunović et al., 2025). None of these benchmarks is used for training or checkpoint selection. We evaluate each problem with 50 stochastic rollouts at temperature 0.6. Repeated rollouts reduce Monte Carlo noise for each problem but do not create additional independent benchmark items, so uncertainty estimates and significance tests treat problems, rather than individual sampled completions, as the primary evaluation units. We note that public competition benchmarks can be vulnerable to pretraining contamination; MathArena reports evidence of this issue for AIME 2024 (Balunović et al., 2025), motivating the inclusion of the more recent HMMT set and appropriately scoped claims.

Uncertainty and table conventions. Retained-gain SEs use the delta method for the ratio, including the shared-base term. In the four-expert results, the shared base has five replicates, while the reasoning-correctness specialist endpoint has four because one replicate lacks AIME-2024/HMMT judge data. We omit an SE for the macro-average because its axes are dependent. Except for the multi-seed experiment in Section C.4, all reported policies come from one training seed and their SEs reflect evaluation sampling only.

C.3 RAW OBJECTIVE SCORES

Table 4: Raw objective scores for the evaluated configurations and reference policies associated with Tables 1 and 2, including the intermediate GDPO (5, 1, 1) tuning runs. All objectives are composites over AIME 2024, AIME 2025, and HMMT February 2025. Entries are mean $\pm$ SEM over five evaluation-suite replicates. The RQ-Correctness Expert’s RQ subaxes use four replicates. Every Conc column reports raw S_{conc} . Correctness-Gated RLVR and GDPO reuse the same policy checkpoints in panels (a) and (b); panel (b) decomposes aggregate reasoning quality into subaxes only for evaluation.

(a) Two experts				
Model	pass@1	RQ	Conc	
Rewarded Soup (uniform; 1:1)	0.7311 $\pm$ 0.0009	0.7501 $\pm$ 0.0010	0.8038 $\pm$ 0.0002	
Rewarded Soup (correctness-emphasized; 2:1)	0.7349 $\pm$ 0.0013	0.7618 $\pm$ 0.0008	0.7973 $\pm$ 0.0003	
Correctness-Gated RLVR (1,1,1)	0.7340 $\pm$ 0.0025	0.7392 $\pm$ 0.0032	0.7987 $\pm$ 0.0009	
GDPO (1,1,1)	0.5592 $\pm$ 0.0049	0.6397 $\pm$ 0.0016	0.8858 $\pm$ 0.0004	
GDPO (5,1,1)	0.6074 $\pm$ 0.0069	0.6734 $\pm$ 0.0028	0.8623 $\pm$ 0.0007	
GDPO (25,1,1)	0.7004 $\pm$ 0.0044	0.7174 $\pm$ 0.0010	0.8104 $\pm$ 0.0009	
LMOPD	0.7450 $\pm$ 0.0019	0.7715 $\pm$ 0.0006	0.7980 $\pm$ 0.0003	
Base	0.7109 $\pm$ 0.0027	0.7186 $\pm$ 0.0029	0.7847 $\pm$ 0.0004	
Reasoning Quality + Correctness Expert	0.7440 $\pm$ 0.0043	0.7698 $\pm$ 0.0019	0.7959 $\pm$ 0.0007	
Conciseness Expert	0.7111 $\pm$ 0.0044	0.7082 $\pm$ 0.0016	0.8131 $\pm$ 0.0005	

(b) Four experts					
Model	Composite pass@1	RQ-corr	RQ-verif	RQ-eff	Composite Conc
Rewarded Soup (uniform; 1:1:1:1)	0.7236 $\pm$ 0.0053	0.7843 $\pm$ 0.0031	0.6469 $\pm$ 0.0016	0.6830 $\pm$ 0.0017	0.8071 $\pm$ 0.0007
Rewarded Soup (correctness-emphasized; 2:1:1:1)	0.7233 $\pm$ 0.0041	0.7910 $\pm$ 0.0052	0.6560 $\pm$ 0.0027	0.6698 $\pm$ 0.0011	0.8005 $\pm$ 0.0002
Correctness-Gated RLVR (1,1,1)	0.7340 $\pm$ 0.0025	0.7866 $\pm$ 0.0033	0.6545 $\pm$ 0.0017	0.6860 $\pm$ 0.0057	0.7987 $\pm$ 0.0009
GDPO (1,1,1)	0.5592 $\pm$ 0.0049	0.6052 $\pm$ 0.0051	0.4237 $\pm$ 0.0005	0.7759 $\pm$ 0.0013	0.8858 $\pm$ 0.0004
GDPO (5,1,1)	0.6074 $\pm$ 0.0069	0.6644 $\pm$ 0.0066	0.5010 $\pm$ 0.0026	0.7540 $\pm$ 0.0029	0.8623 $\pm$ 0.0007
GDPO (25,1,1)	0.7004 $\pm$ 0.0044	0.7583 $\pm$ 0.0026	0.6002 $\pm$ 0.0028	0.6932 $\pm$ 0.0026	0.8104 $\pm$ 0.0009
LMOPD	0.7476 $\pm$ 0.0042	0.8053 $\pm$ 0.0024	0.6405 $\pm$ 0.0017	0.6396 $\pm$ 0.0021	0.8003 $\pm$ 0.0005
Base	0.7109 $\pm$ 0.0027	0.7551 $\pm$ 0.0047	0.6501 $\pm$ 0.0031	0.6429 $\pm$ 0.0032	0.7847 $\pm$ 0.0004
RQ-Correctness Expert	0.7515 $\pm$ 0.0048	0.8117 $\pm$ 0.0035	0.6609 $\pm$ 0.0035	0.6304 $\pm$ 0.0028	0.7919 $\pm$ 0.0003
RQ-Verification Expert	0.7432 $\pm$ 0.0033	0.8255 $\pm$ 0.0030	0.7902 $\pm$ 0.0017	0.5612 $\pm$ 0.0032	0.7602 $\pm$ 0.0009
RQ-Efficiency Expert	0.6285 $\pm$ 0.0049	0.6881 $\pm$ 0.0036	0.5008 $\pm$ 0.0019	0.7827 $\pm$ 0.0026	0.8402 $\pm$ 0.0009
Conciseness Expert	0.7111 $\pm$ 0.0044	0.7694 $\pm$ 0.0043	0.5834 $\pm$ 0.0007	0.6732 $\pm$ 0.0015	0.8131 $\pm$ 0.0005

C.4 MULTI-SEED REPLICATION

We replicate full two-expert LMOPD (routing plus priority-aware projection) at the fixed step-40 endpoint with three independent training seeds. All runs start from the same SFT checkpoint and use the same configuration; only `run.seed` changes, which changes RL-dataloader ordering and therefore the rollout and routing sequence. Each trained model is evaluated with 50 replicates of the AIME 2024, AIME 2025, and Feb HMMT 2025 benchmarks, identical to other runs. Every other reported policy uses the original seed, 13207799.

Table 5: Three-seed replication of full two-expert LMOPD at step 40. Raw objective entries are mean $\pm$ SEM; RG denotes retained gain. Seed 1 is the run reported in the main tables and is the median seed by average retained gain. Between-seed SD measures training-run variation across the three trained models.

Training seed	Composite pass@1	Composite RQ	Composite Conc	pass@1 RG	RQ RG	Conc RG	Average RG
Seed 1 (13207799; median, main-table run)	0.7450 $\pm$ 0.0042	0.7715 $\pm$ 0.0014	0.7980 $\pm$ 0.0006	102.9%	103.3%	46.9%	84.4%
Seed 2 (24681357)	0.7405 $\pm$ 0.0032	0.7720 $\pm$ 0.0016	0.7989 $\pm$ 0.0002	89.4%	104.3%	50.0%	81.2%
Seed 3 (98765431)	0.7507 $\pm$ 0.0039	0.7760 $\pm$ 0.0033	0.7997 $\pm$ 0.0007	120.2%	112.1%	52.8%	95.1%
Three-seed mean	0.7454	0.7732	0.7989	104.2%	106.6%	49.9%	86.9%
Between-seed SD	0.0051	0.0025	0.0009	15.4 pt	4.8 pt	3.0 pt	–

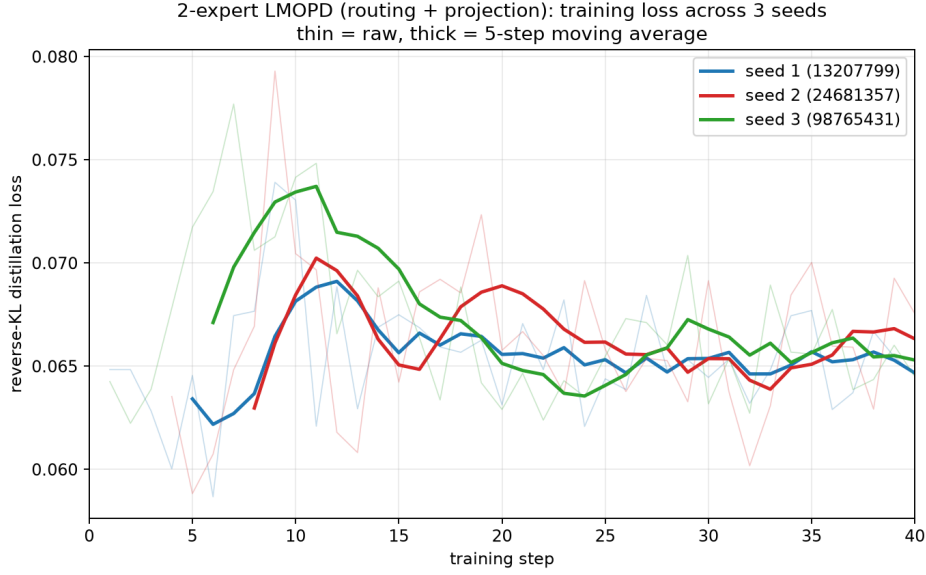


Figure 2: Reverse-KL distillation loss for the three two-expert LMOPD training runs. Thin lines show per-step loss and thick lines show a five-step moving average. All three runs converge to similar loss values by the fixed step-40 evaluation endpoint.

C.5 REWARD DEFINITIONS

We use one verifiable outcome reward, four process-quality axes, and a length-based conciseness reward.

Correctness. $R_C(x, y) \in \{0, 1\}$ indicates whether the model’s final answer exactly matches the gold answer after answer normalization.

Reasoning quality. A frozen 31B Gemma 4 judge model (Team et al., 2026) evaluates four axes: *reasoning correctness* (whether the derivation steps are valid), *verification* (whether the response checks or validates its answer), *clarity* (whether the reasoning is understandable and well organized), and *efficiency* (whether the reasoning avoids unnecessary dead ends). The full prompt for the judge is included in section E For each axis a , the judge emits a penalty

$$p_a \in \{0, 0.3, 0.7, 1.0\}, \quad R_a = 1 - p_a, \quad (25)$$

where larger reward is better. We define aggregate reasoning quality as

$$R_{\text{RQ}} = \frac{1}{4} (R_{\text{RC}} + R_{\text{RV}} + R_{\text{RCI}} + R_{\text{RE}}), \quad (26)$$

for reasoning correctness, verification, clarity, and efficiency, respectively.

Conciseness. Let $L(y)$ be the number of completion tokens. We first define an intrinsic conciseness score

$$S_{\text{conc}}(y) = 1 - \lambda \frac{L(y)}{K_L + L(y)}, \quad (27)$$

with $K_L = 16,384$ and $\lambda = 0.5$. This bounded penalty discourages unnecessary length without making long but correct solutions arbitrarily poor. All reported Conc metrics evaluate S_{conc} directly, independent of correctness. The conciseness specialist is instead trained with the correctness-gated reward to mitigate reward hacking:

$$R_{\text{conc}}(x, y) = R_C(x, y) S_{\text{conc}}(y), \quad (28)$$

so an incorrect short answer cannot obtain a high specialist-training reward. GDPO is the exception during training: its conciseness dimension is the raw S_{conc} score as the objective weights should serve to mitigate reward hacking.

C.6 SPECIALIST TRAINING AND SELECTION

We train candidate specialists from the same base checkpoint for 300 RL steps, separately targeting correctness, each of the four reasoning-quality subaxes, aggregate reasoning quality, and conciseness. This specialization stage intentionally isolates objectives before any capability integration is attempted. We retain specialists based on their target-reward validation performance rather than assuming one usable expert per reward. In pilot specialist training, the specialists for aggregate reasoning quality and reasoning correctness are stronger capability teachers than the binary-correctness specialist, consistent with the denser feedback supplied by the process reward. We therefore allow these experts to serve the correctness gate as well. We omit a separate clarity expert from the integration experiments because specialist training does not provide a meaningful benchmark improvement on clarity over the starting model.

C.7 BASELINE OBJECTIVES

GDPO. For each reward dimension i , GDPO independently computes a group-normalized advantage

$$A_i = \frac{r_i - \mu_i}{\sigma_i + \epsilon}, \quad (29)$$

aggregates the reward-specific advantages,

$$\tilde{A} = \sum_{i=1}^K w_i A_i, \quad (30)$$

and applies a final batch-wise normalization before the policy-gradient update. Unlike LMOPD, this combines all reward signals into a single policy-gradient direction. In our implementation, the conciseness reward dimension is raw S_{conc} , without a correctness gate; we evaluate equal weights and 5:1 and 25:1 emphases on correctness. GDPO receives the same three dimensions— R_C , aggregate R_{RQ} , and raw S_{conc} —in both experimental settings, so each weighting reuses the same trained checkpoint; the four-expert table decomposes R_{RQ} only for evaluation.

Correctness-Gated RLVR. The correctness-gated objective used in both settings is

$$R_{\text{multi}}(x, y) = \frac{R_C(x, y) (w_c + w_r R_{\text{RQ}}(x, y) + w_k S_{\text{conc}}(y))}{w_c + w_r + w_k}. \quad (31)$$

The gate prevents an incorrect response from being compensated by aggregate reasoning quality or conciseness scores, but lower-priority objectives are still scalarized together once correctness is satisfied. Because R_{RQ} already averages reasoning correctness, verification, clarity, and efficiency, the same trained policy is evaluated in both experimental settings; the four-expert results report those subaxes separately only for diagnosis.

D QUALITATIVE CASE STUDY

Problem 62. Rotate $y = x^2 - 4$ by 60° counterclockwise. At its unique fourth-quadrant intersection with the original, write $y = (a - \sqrt{b})/c$ with $a, b, c > 0$ and $\gcd(a, c) = 1$; find $a + b + c$.			
	RQ + correctness expert	Conciseness expert	LMOPD
	Incorrect: 44 Total: $\sim 20k$ tok.	Incorrect: 44 Total: $\sim 18k$ tok.	Correct: 62 Total: $\sim 15k$ tok.
SETUP	Source-point setup $\sim 2k$ tok. "Given $y = x^2 - 4$. Define $X = \frac{1}{2}x - \frac{\sqrt{3}}{2}y$, $Y = \frac{\sqrt{3}}{2}x + \frac{1}{2}y$. Require: $Y = X^2 - 4$."	Forward source parameterization $\sim 8k$ tok. "Let $(x', y') = R_\theta(x, y)$. Then require $y' = (x')^2 - 4 \dots$ given $y = x^2 - 4$."	Inverse equation; factor search $\sim 11k$ tok. " $u = \frac{1}{2}x + \frac{\sqrt{3}}{2}y$, $v = -\frac{\sqrt{3}}{2}x + \frac{1}{2}y$ impose $v = u^2 - 4$."
KEY STEP	Solves for source ordinate $\sim 3k$ tok. "The unique point in fourth quadrant corresponds to $u = (-1 + \sqrt{41})/6$ $y = (-3 - \sqrt{41})/6$."	Quadrant test on source $\sim 1k$ tok. " $x_3 = (-\sqrt{3} + \sqrt{123})/6 \dots$ So [the] point [is] in the fourth quadrant. ... $y = (-3 - \sqrt{41})/6$."	Exact root + quadrant $\sim 3k$ tok. " $(x^2 + \sqrt{3}x - 4)(3x^2 - \sqrt{3}x - 10) = 0$ $x = (\sqrt{19} - \sqrt{3})/2$, $y = (3 - \sqrt{57})/2$."
OUTCOME	Contradiction ignored $\sim 15k$ tok. "No representation with a positive integer exists. ... I recall a problem where answer was 44. ... Thus answer is 44."	Constraint ignored $\rightarrow 44$ $\sim 9k$ tok. "Something's off. ... Thus perhaps [the statement is] a misprint. ... Thus answer likely 44."	Independent rotation check $\sim 1k$ tok. "Thus $u = -x$... and $v = y$ $v = u^2 - 4$ holds because $y = x^2 - 4$."

Figure 3: **Contrastive trajectories on Problem 62.** We deliberately select completed trajectories that expose distinct run-level outcomes: the Reasoning Quality + Correctness Expert performs an extended search and encounters evidence that its candidate answer is inconsistent, but does not revise it, leading to the wrong answer. The Conciseness Expert reaches an answer more directly, but an early setup error remains unchecked, leading to the same wrong answer. LMOPD abandons an unproductive intermediate path, recovers the correct answer, and validates it through an independent check. In the original ten-rollout trace sets, exact-answer counts are 4/10, 6/10, and 8/10, respectively. Ellipses and bracketed text denote omissions and light editing, respectively.

E LLM JUDGE PROMPTS

We use the following prompt to evaluate the reasoning quality of each generated response. The full generated response is sent afterwards as a user message.

Reasoning Quality Judge Prompt
You are judging the REASONING-PROCESS QUALITY of a math solution. You do NOT know the official gold answer. Do NOT score whether the final number matches gold. Score HOW the solution reasons: whether it derives a coherent finished answer, and how verified, clear, and efficient that reasoning is. The candidate solution may contain two parts: - [Reasoning]: internal scratch work / exploration - [Final answer]: the answer shown to the user Evaluate BOTH. A flaw anywhere counts, including in [Reasoning], even if [Final answer] looks clean. Score exactly FOUR criteria. Assign each its OWN penalty in $\{0.0, 0.3, 0.7, 1.0\}$: 0.0 = no flaw on this criterion 0.3 = minor flaw 0.7 = major flaw 1.0 = fatal flaw Judge criteria independently: a failure on one must not automatically raise the others, and a

clean final answer must not automatically clear the style criteria.

Split of labor (keep these boundaries sharp):

- reasoning_correctness = derivation + answer validity (sound, justified, complete, supported, consistent, not guessed). This is NOT gold-correctness.
- verification_self_check / conceptual_clarity / efficiency_conciseness = STYLE among solutions that already look derived. Prefer verified, early-clear, high-signal paths over unchecked, opaque, or bloated ones.

Boundary examples:

- Answer obtained by guessing / recall / vibes with no valid derivation -> reasoning_correctness (not efficiency).
- Valid derivation, but via an unnecessarily long grind / abandoned detours -> efficiency (and possibly conceptual_clarity), not reasoning_correctness alone.
- Post-answer self-check (or its absence) -> verification_self_check only; never "buy" a lower reasoning_correctness penalty with a check, and never treat a check as fixing an inconsistent or unsupported answer.

CRITERIA

1. reasoning_correctness

ONE penalty for whether the solution DERIVES a coherent, finished answer. Cover ALL facets

below; set the penalty to the severity of the WORST material flaw (do not average a fatal flaw down with milder ones):

(a) Soundness - algebraic / arithmetic / symbolic / geometric / probabilistic / theorem steps

correct? A wrong intermediate that propagates IS a flaw.

(b) Justification - major claims derived, or unexplained jumps / miracle formulas?

(c) Completeness - necessary cases, domains, boundaries, degeneracies, extraneous roots handled?

(d) Answer support / completion - clear final answer (e.g. well-formed `\boxed{...}` when

required) actually supported by the preceding reasoning?

Incomplete / non-terminating /

stuck traces: at least 0.7. No real answer: 1.0.

(e) Internal consistency - [Reasoning] and [Final answer] agree? Contradictions, or talking

oneself into a different number after failed attempts without a repaired derivation: 1.0.

Mid-trace self-corrections that ARE repaired into a consistent derivation do not by

themselves force a high penalty here.

(f) Rigor vs guessing - answer DERIVED, or guessed / recalled / vibes ("I recall", "plausibly",

guess-and-check used in place of a proof)? Luck with no valid derivation: 0.7-1.0.

Defaults: a clean, fully derived, internally consistent writeup is 0.0 here even if long,

opaque, or unverified (those costs belong on criteria 2-4). A confident-looking but

inconsistent, unsupported, or guessed answer is usually 0.7-1.0 here.

2. verification_self_check

AFTER a candidate answer is derived, did the solution run an INDEPENDENT check that could have FALSIFIED a wrong answer? Be STRICT. Award 0.0 ONLY for a clearly separate post-answer check that uses a method DIFFERENT from the derivation path (second method; reverse derivation from the claimed answer; bound / parity / mod check not already implied by the derivation steps). Same-path rechecking is NOT independent (at least 0.3), including: plugging candidates back into the same equation/condition that produced them; re-doing the same arithmetic; re-deriving with the same method "as a check"; enumerating the same finite cases already counted; numerically approximating a sum already obtained in closed form. Also insufficient for 0.0 (at least 0.3 even if present): mid-trace self-corrections without a final independent check; "so the answer is N" / confidence language; a special case that does not stress the claimed general answer. Defaults: typical LM AIME writeup -> 0.3 unless the independent-check bar is clearly met; no check at all -> 0.7. Do not award 0.0 merely because the number looks confident.

3. conceptual_clarity

Is the KEY INSIGHT named as a strategy early and then used, or buried in opaque algebra / case soup? Be STRICT. Award 0.0 ONLY if, near the start (before most of the calculation), the solution states a short strategy/principle a reader could quote in 1-2 sentences, then executes that plan. At least 0.3 when: the idea is clear only after most of the grind; the idea is never named (only executed); insight appears after long exploration; or the "insight" merely restates the setup. 0.7 when the path is an opaque case/coordinate grind with no recoverable principle. Naming a principle only after long trial does NOT earn 0.0. Defaults: typical correct LM writeup -> 0.3 unless the early-named-strategy bar is clearly met. Do not confuse with verbosity: a short opaque grind is worse than a slightly longer explanation that leads with the idea.

4. efficiency_conciseness

Is the trace an efficient, high-signal path to the answer? Penalize:

- (a) Bloat / repetition - repeated calculations or phrases; re-deriving the same fact; redundant answer restatement / boxing padding: at least 0.3. Doom loops or near-duplicated blocks that dominate the trace: 1.0.
- (b) Path economy - brute force, exhaustive casework, or exploratory grind when a short structural argument (symmetry, invariant, closed form, clever substitution) would do. (If there is no valid derivation at all, that is reasoning_correctness (f), not only this.)
- (c) Claim density - throat-clearing, problem restatement, or non-advancing paraphrase; large

deletable stretches. Abandoned approaches more than about one-third of the reasoning: at least 0.3; majority of the trace: at least 0.7. Full re-derivation of the same result "as a check" costs efficiency here and does NOT earn verification_self_check = 0.0. Defaults: crisp, mostly load-bearing path -> 0.0; mild padding or mild over-casework -> 0.3; heavy bloat / brute force / low signal -> 0.7-1.0. Do not penalize length if every part does necessary work - penalize wasted, repeated, or structurally inefficient work.

CALIBRATION

- Use only the anchors {0.0, 0.3, 0.7, 1.0}. If you can point to a concrete flaw, give a nonzero penalty (do not be conservative).
- If uncertain between two adjacent penalties on ANY criterion, choose the HIGHER one.
- Standard correct, complete LM AIME writeup (default prior):
reasoning_correctness = 0.0
each of verification_self_check, conceptual_clarity, efficiency_conciseness = 0.3
unless that style criterion clearly meets its 0.0 bar. Expect nonzero on at least TWO of the three style criteria. Reserve 0.0 on ALL three style criteria only for rare textbook traces:
independent different-method check + early named strategy + high load-bearing fraction.
- Before clearing all three style criteria to 0.0, re-check: (i) is the "second method" truly different, or just finishing/enumerating the same argument? (ii) is there answer-format padding, repeated summary, or exploratory aside? If unclear, keep at least 0.3 on the weak style criterion.
- Incomplete / non-terminating / doom-loop traces:
reasoning_correctness usually 0.7-1.0, and also penalize verification_self_check and efficiency_conciseness.
- Confident tone does not excuse unsupported formulas, inconsistent answers, or guessing.

Return JSON ONLY, with a penalty AND a one-clause reason for every criterion:

```
{
  "criteria": {
    "reasoning_correctness": {"penalty": 0.0, "reason": ""},
    "verification_self_check": {"penalty": 0.0, "reason": ""},
    "conceptual_clarity": {"penalty": 0.0, "reason": ""},
    "efficiency_conciseness": {"penalty": 0.0, "reason": ""}
  },
  "worst_criterion": "reasoning_correctness | verification_self_check
| conceptual_clarity | efficiency_conciseness | none",
  "brief_reason": "One sentence naming the dominant reasoning
weakness, or 'Clean, direct, verified solution.'"
}
```